

EXPERIMENT – 3

AIM: To design and implement an Employee–Department–Project database schema and perform SQL queries using Selection, Projection, Aggregate Functions, GROUP BY, HAVING, CASE expressions, and ORDER BY.

SOURCE CODE:

DATABASE CREATING

```
CREATE DATABASE EmployeeManagement;  
USE EmployeeManagement;
```

TABLE CREATING

-- Department Table

```
CREATE TABLE Department (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50) NOT NULL,  
    location VARCHAR(50)  
);
```

-- Employee Table

```
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50) NOT NULL,  
    gender CHAR(1),  
    salary DECIMAL(10,2),  
    hire_date DATE,  
    dept_id INT,  
    FOREIGN KEY (dept_id)  
        REFERENCES Department(dept_id)  
);
```

-- Project Table

```
CREATE TABLE Project (  
    project_id INT PRIMARY KEY,  
    project_name VARCHAR(100) NOT NULL,  
    budget DECIMAL(12,2),  
    dept_id INT,  
    FOREIGN KEY (dept_id)  
        REFERENCES Department(dept_id)  
);
```

-- Employee-Project Relationship

```
CREATE TABLE Employee_Project (  
    emp_id INT,  
    project_id INT,  
    hours_worked INT,  
  
    PRIMARY KEY (emp_id, project_id),  
  
    FOREIGN KEY (emp_id)  
        REFERENCES Employee(emp_id),  
  
    FOREIGN KEY (project_id)  
        REFERENCES Project(project_id)  
);
```

INSERTING DEPARTMENTS

```
INSERT INTO Department (dept_id, dept_name, location)  
VALUES  
(1, 'IT', 'Delhi'),  
(2, 'HR', 'Mumbai'),  
(3, 'Finance', 'Bangalore'),  
(4, 'Marketing', 'Pune'),  
(5, 'Research', 'Hyderabad');
```

INSERTING 30 EMPLOYEES

```
INSERT INTO Employee  
(emp_id, emp_name, gender, salary, hire_date, dept_id)  
VALUES  
(101, 'Aarav Sharma', 'M', 75000, '2021-01-15', 1),  
(102, 'Ananya Singh', 'F', 68000, '2022-03-10', 1),  
(103, 'Rohan Verma', 'M', 82000, '2020-07-20', 1),  
(104, 'Priya Gupta', 'F', 72000, '2021-11-05', 1),  
(105, 'Karan Mehta', 'M', 95000, '2019-06-18', 1),  
(106, 'Sneha Kapoor', 'F', 64000, '2023-01-12', 1),  
(107, 'Rahul Kumar', 'M', 55000, '2022-05-15', 2),  
(108, 'Neha Sharma', 'F', 62000, '2021-09-22', 2),  
(109, 'Amit Jain', 'M', 58000, '2020-04-11', 2),  
(110, 'Pooja Verma', 'F', 67000, '2023-02-19', 2),  
(111, 'Vikas Singh', 'M', 60000, '2022-08-01', 2),  
(112, 'Riya Agarwal', 'F', 71000, '2019-12-10', 2),  
(113, 'Arjun Patel', 'M', 88000, '2020-01-25', 3),  
(114, 'Meera Shah', 'F', 92000, '2019-08-14', 3),  
(115, 'Suresh Rao', 'M', 76000, '2021-06-30', 3),  
(116, 'Kavya Iyer', 'F', 85000, '2022-10-17', 3),  
(117, 'Manish Gupta', 'M', 73000, '2023-03-21', 3),
```

```
(118, 'Nisha Reddy', 'F', 90000, '2020-11-09', 3),
(119, 'Aditya Joshi', 'M', 70000, '2021-02-28', 4),
(120, 'Simran Kaur', 'F', 66000, '2022-04-15', 4),
(121, 'Mohit Bansal', 'M', 74000, '2020-09-12', 4),
(122, 'Isha Malhotra', 'F', 81000, '2019-07-07', 4),
(123, 'Nikhil Sethi', 'M', 69000, '2023-01-05', 4),
(124, 'Tanya Arora', 'F', 77000, '2021-12-18', 4),
(125, 'Dev Malhotra', 'M', 98000, '2018-05-20', 5),
(126, 'Aditi Mishra', 'F', 91000, '2020-03-16', 5),
(127, 'Varun Nair', 'M', 87000, '2021-10-08', 5),
(128, 'Shreya Das', 'F', 93000, '2019-11-25', 5),
(129, 'Yash Thakur', 'M', 79000, '2022-07-14', 5),
(130, 'Diya Kapoor', 'F', 96000, '2023-02-02', 5);
```

INSERTING 8 PROJECTS

```
INSERT INTO Project
(project_id, project_name, budget, dept_id)
VALUES
(201, 'AI Chatbot', 500000, 1),
(202, 'Cloud Migration', 750000, 1),
(203, 'Employee Training', 250000, 2),
(204, 'Financial Analysis', 600000, 3),
(205, 'Digital Marketing', 450000, 4),
(206, 'Market Research', 350000, 4),
(207, 'Machine Learning Research', 900000, 5),
(208, 'AI Security System', 1000000, 5);
```

ASSIGNING EMPLOYEES TO PROJECTS

```
INSERT INTO Employee_Project
(emp_id, project_id, hours_worked)
VALUES
(101, 201, 120),
(102, 201, 100),
(103, 202, 150),
(104, 202, 110),
(105, 201, 130),
(106, 202, 90),
(107, 203, 100),
(108, 203, 120),
(109, 203, 80),
(110, 203, 110),
(111, 203, 90),
(112, 203, 130),
(113, 204, 140),
(114, 204, 150),
(115, 204, 120),
(116, 204, 135),
```

```
(117, 204, 100),
(118, 204, 145),
(119, 205, 110),
(120, 205, 100),
(121, 206, 120),
(122, 205, 140),
(123, 206, 90),
(124, 206, 130),
(125, 207, 160),
(126, 207, 140),
(127, 208, 150),
(128, 208, 160),
(129, 207, 120),
(130, 208, 135);
```

SQL QUERIES:

SELECTION

```
SELECT *
FROM Employee
WHERE salary > 80000;
```

OUTPUT:

MySQL - db_454577wwy

EXPLORER

Search...

TABLES

- demo
- department
- employee
- employee_project
- project

```

142
143 SELECT *
144 FROM Employee
145 WHERE salary > 80000;
146
147
148

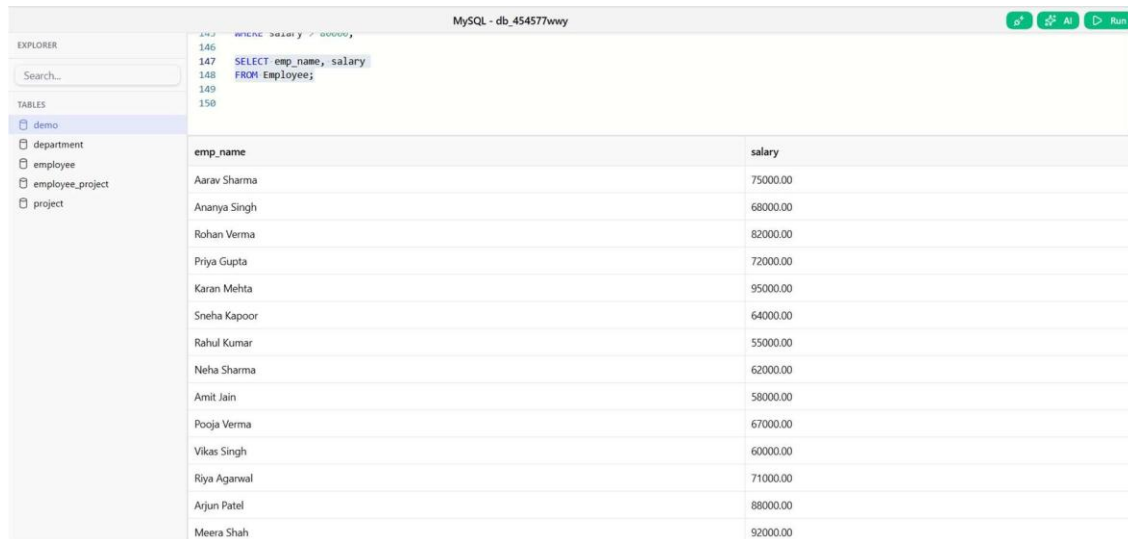
```

emp_id	emp_name	gender	salary	hire_date	dept_id
103	Rohan Verma	M	82000.00	2020-07-20T00:00:00.000Z	1
105	Karan Mehta	M	95000.00	2019-06-18T00:00:00.000Z	1
113	Arjun Patel	M	88000.00	2020-01-25T00:00:00.000Z	3
114	Meera Shah	F	92000.00	2019-08-14T00:00:00.000Z	3
116	Kavya Iyer	F	85000.00	2022-10-17T00:00:00.000Z	3
118	Nisha Reddy	F	90000.00	2020-11-09T00:00:00.000Z	3
122	Isha Malhotra	F	81000.00	2019-07-07T00:00:00.000Z	4
125	Dev Malhotra	M	98000.00	2018-05-20T00:00:00.000Z	5
126	Aditi Mishra	F	91000.00	2020-03-16T00:00:00.000Z	5
127	Varun Nair	M	87000.00	2021-10-08T00:00:00.000Z	5
128	Shreya Das	F	93000.00	2019-11-25T00:00:00.000Z	5
130	Diya Kapoor	F	96000.00	2023-02-02T00:00:00.000Z	5

PROJECTION

```
SELECT emp_name, salary
FROM Employee;
```

OUTPUT:



The screenshot shows the MySQL Workbench interface. On the left, the 'EXPLORER' pane lists the 'demo' database and its tables: 'department', 'employee', 'employee_project', and 'project'. The 'demo' database is selected. The main editor shows the following SQL query:

```
146 SELECT emp_name, salary
147 FROM Employee;
```

The output of the query is displayed in a table with two columns: 'emp_name' and 'salary'.

emp_name	salary
Aarav Sharma	75000.00
Ananya Singh	68000.00
Rohan Verma	82000.00
Priya Gupta	72000.00
Karan Mehta	95000.00
Sneha Kapoor	64000.00
Rahul Kumar	55000.00
Neha Sharma	62000.00
Amit Jain	58000.00
Pooja Verma	67000.00
Vikas Singh	60000.00
Riya Agarwal	71000.00
Arjun Patel	88000.00
Meera Shah	92000.00

AGGREGATE FUNCTIONS

```
SELECT
    COUNT(*) AS total_employees,
    SUM(salary) AS total_salary,
    AVG(salary) AS average_salary,
    MAX(salary) AS highest_salary,
    MIN(salary) AS lowest_salary
FROM Employee;
```

OUTPUT:



The screenshot shows the MySQL Workbench interface. On the left, the 'EXPLORER' pane lists the 'demo' database and its tables: 'department', 'employee', 'employee_project', and 'project'. The 'demo' database is selected. The main editor shows the following SQL query:

```
149 SELECT
150     COUNT(*) AS total_employees,
151     SUM(salary) AS total_salary,
152     AVG(salary) AS average_salary,
153     MAX(salary) AS highest_salary,
154     MIN(salary) AS lowest_salary
155 FROM Employee;
```

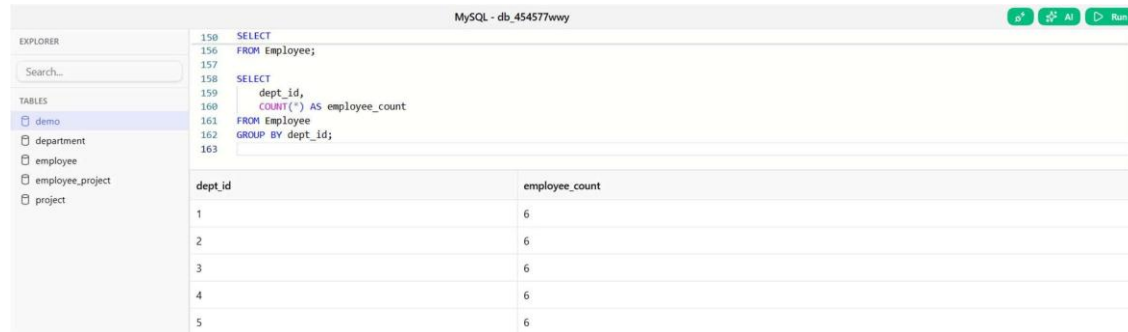
The output of the query is displayed in a table with five columns: 'total_employees', 'total_salary', 'average_salary', 'highest_salary', and 'lowest_salary'.

total_employees	total_salary	average_salary	highest_salary	lowest_salary
30	2314000.00	77133.333333	98000.00	55000.00

GROUP BY

```
SELECT
    dept_id,
    COUNT(*) AS employee_count
FROM Employee
GROUP BY dept_id;
```

OUTPUT:



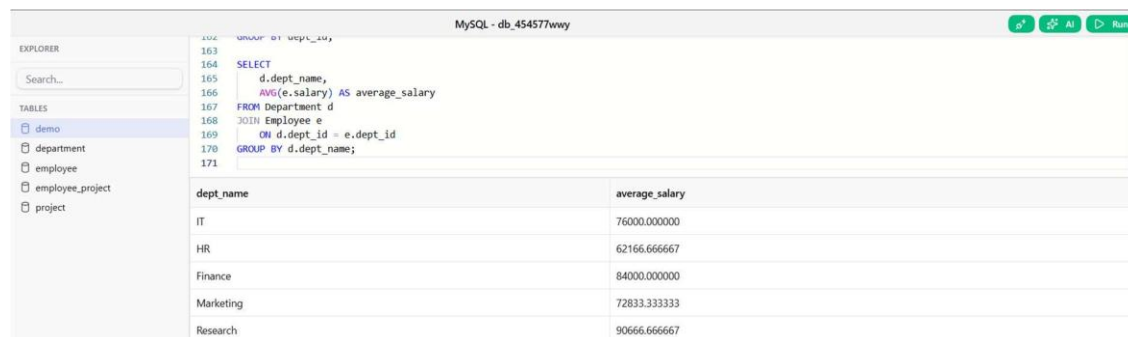
The screenshot shows a MySQL IDE window titled "MySQL - db_454577wwy". On the left is an "EXPLORER" pane with a search bar and a "TABLES" list containing "demo", "department", "employee", "employee_project", and "project". The "demo" table is selected. The main editor shows a SQL query with line numbers 150 to 163. The query is:
150 SELECT
151 FROM Employee;
152
153 SELECT
154 dept_id,
155 COUNT(*) AS employee_count
156 FROM Employee
157 GROUP BY dept_id;
158
159
160
161
162
163
Below the query, the output is displayed as a table with two columns: "dept_id" and "employee_count". The output shows five rows, all with a count of 6.

dept_id	employee_count
1	6
2	6
3	6
4	6
5	6

GROUP BY WITH JOIN

```
SELECT
    d.dept_name,
    AVG(e.salary) AS average_salary
FROM Department d
JOIN Employee e
    ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

OUTPUT:



The screenshot shows a MySQL IDE window titled "MySQL - db_454577wwy". On the left is an "EXPLORER" pane with a search bar and a "TABLES" list containing "demo", "department", "employee", "employee_project", and "project". The "demo" table is selected. The main editor shows a SQL query with line numbers 164 to 171. The query is:
164 SELECT
165 d.dept_name,
166 AVG(e.salary) AS average_salary
167 FROM Department d
168 JOIN Employee e
169 ON d.dept_id = e.dept_id
170 GROUP BY d.dept_name;
171
Below the query, the output is displayed as a table with two columns: "dept_name" and "average_salary". The output shows five rows with department names and their average salaries.

dept_name	average_salary
IT	76000.000000
HR	62166.666667
Finance	84000.000000
Marketing	72833.333333
Research	90666.666667

HAVING

```
SELECT
    d.dept_name,
    AVG(e.salary) AS average_salary
FROM Department d
JOIN Employee e
    ON d.dept_id = e.dept_id
GROUP BY d.dept_name
HAVING AVG(e.salary) > 75000;
```

OUTPUT:



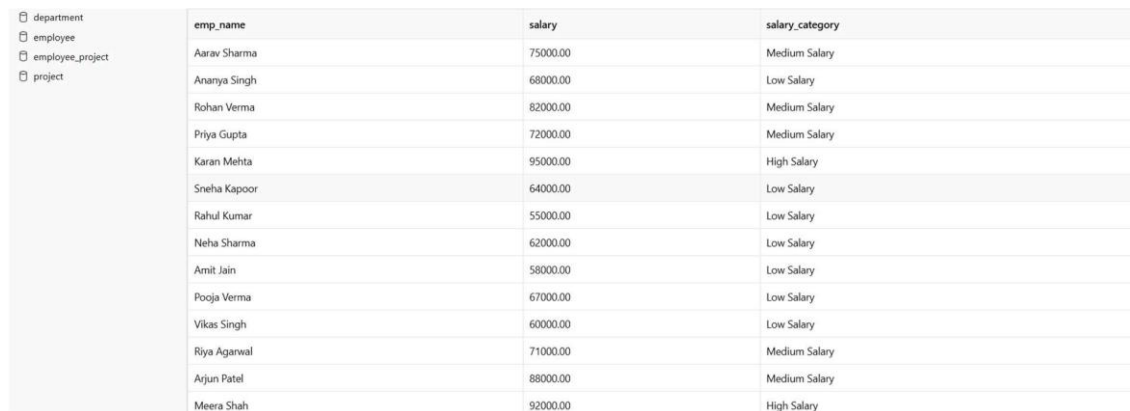
The screenshot shows a MySQL IDE window titled 'MySQL - db_454577wwy'. The left sidebar contains an 'EXPLORER' pane with a search bar and a 'TABLES' list including 'demo', 'department', 'employee', 'employee_project', and 'project'. The main editor displays a SQL query with line numbers 172 to 180. The query is identical to the one in the previous block. Below the editor, the query results are displayed in a table with two columns: 'dept_name' and 'average_salary'.

dept_name	average_salary
IT	76000.000000
Finance	84000.000000
Research	90666.666667

CASE EXPRESSIONS

```
SELECT
    emp_name,
    salary,
    CASE
        WHEN salary >= 90000 THEN 'High Salary'
        WHEN salary >= 70000 THEN 'Medium Salary'
        ELSE 'Low Salary'
    END AS salary_category
FROM Employee;
```

OUTPUT:



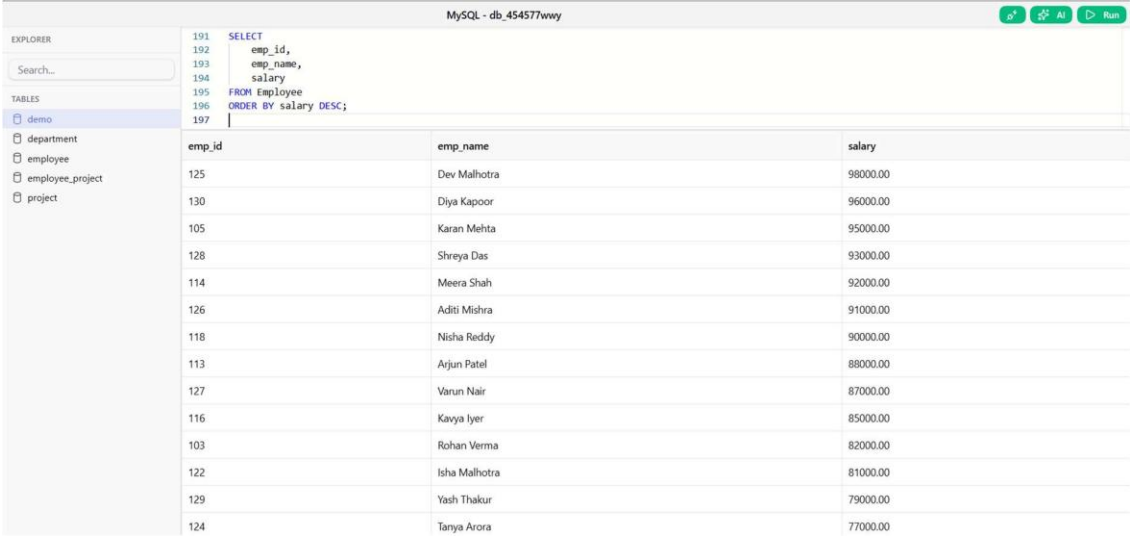
The screenshot shows a MySQL IDE window with the same sidebar as before. The main editor displays a SQL query using a CASE expression to categorize salaries. The query results are displayed in a table with four columns: 'emp_name', 'salary', and 'salary_category'.

emp_name	salary	salary_category
Aarav Sharma	75000.00	Medium Salary
Ananya Singh	68000.00	Low Salary
Rohan Verma	82000.00	Medium Salary
Priya Gupta	72000.00	Medium Salary
Karan Mehta	95000.00	High Salary
Sneha Kapoor	64000.00	Low Salary
Rahul Kumar	55000.00	Low Salary
Neha Sharma	62000.00	Low Salary
Amit Jain	58000.00	Low Salary
Pooja Verma	67000.00	Low Salary
Vikas Singh	60000.00	Low Salary
Riya Agarwal	71000.00	Medium Salary
Arjun Patel	88000.00	Medium Salary
Meera Shah	92000.00	High Salary

ORDER BY

```
SELECT
    emp_id,
    emp_name,
    salary
FROM Employee
ORDER BY salary DESC;
```

OUTPUT:



The screenshot shows a MySQL IDE window titled "MySQL - db_454577wwy". On the left, the "EXPLORER" pane shows a tree view with "demo" selected, containing "department", "employee", "employee_project", and "project". The main editor displays a SQL query:

```
SELECT
emp_id,
emp_name,
salary
FROM Employee
ORDER BY salary DESC;
```

 Below the query, the results are shown in a table with three columns: "emp_id", "emp_name", and "salary". The table contains 15 rows of data, sorted by salary in descending order.

emp_id	emp_name	salary
125	Dev Malhotra	98000.00
130	Diya Kapoor	96000.00
105	Karan Mehta	95000.00
128	Shreya Das	93000.00
114	Meera Shah	92000.00
126	Aditi Mishra	91000.00
118	Nisha Reddy	90000.00
113	Arjun Patel	88000.00
127	Varun Nair	87000.00
116	Kavya Iyer	85000.00
103	Rohan Verma	82000.00
122	Isha Malhotra	81000.00
129	Yash Thakur	79000.00
124	Tanya Arora	77000.00

COMPLETE EMPLOYEE-DEPARTMENT REPORT

```
SELECT
    e.emp_id,
    e.emp_name,
    d.dept_name,
    e.salary
FROM Employee e
JOIN Department d
    ON e.dept_id = d.dept_id
ORDER BY d.dept_name, e.salary DESC;
```

OUTPUT:

MySQL - db_454577wwy

EXPLORER

Search...

TABLES

demo

department

employee

employee_project

project

198 SELECT

199 e.emp_id,

200 e.emp_name,

201 d.dept_name,

202 e.salary

203 FROM Employee e

204 JOIN Department d

emp_id	emp_name	dept_name	salary
114	Meera Shah	Finance	92000.00
118	Nisha Reddy	Finance	90000.00
113	Arjun Patel	Finance	88000.00
116	Kavya Iyer	Finance	85000.00
115	Suresh Rao	Finance	76000.00
117	Manish Gupta	Finance	73000.00
112	Riya Agarwal	HR	71000.00
110	Pooja Verma	HR	67000.00
108	Neha Sharma	HR	62000.00
111	Vikas Singh	HR	60000.00
109	Amit Jain	HR	58000.00
107	Rahul Kumar	HR	55000.00
105	Karan Mehta	IT	95000.00
103	Rohan Verma	IT	82000.00

PROJECT DETAILS

```
SELECT
    p.project_name,
    d.dept_name,
    p.budget
FROM Project p
JOIN Department d
    ON p.dept_id = d.dept_id
ORDER BY p.budget DESC;
```

OUTPUT:

MySQL - db_454577wwy

EXPLORER

Search...

TABLES

demo

department

employee

employee_project

project

208 SELECT

209 p.project_name,

210 d.dept_name,

211 p.budget

212 FROM Project p

213 JOIN Department d

214 ON p.dept_id = d.dept_id

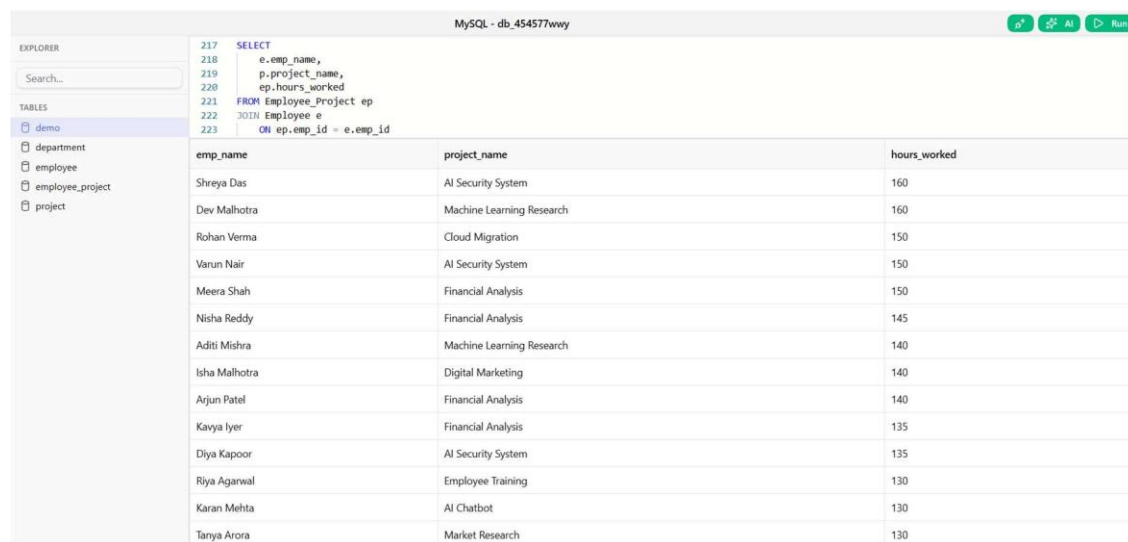
215 ORDER BY p.budget DESC;

project_name	dept_name	budget
AI Security System	Research	1000000.00
Machine Learning Research	Research	900000.00
Cloud Migration	IT	750000.00
Financial Analysis	Finance	600000.00
AI Chatbot	IT	500000.00
Digital Marketing	Marketing	450000.00
Market Research	Marketing	350000.00
Employee Training	HR	250000.00

EMPLOYEES WORKING ON PROJECTS

```
SELECT
    e.emp_name,
    p.project_name,
    ep.hours_worked
FROM Employee_Project ep
JOIN Employee e
    ON ep.emp_id = e.emp_id
JOIN Project p
    ON ep.project_id = p.project_id
ORDER BY ep.hours_worked DESC;
```

OUTPUT:



The screenshot shows a MySQL database interface with a query editor on the left and a results table on the right. The query editor contains the following SQL code:

```
217 SELECT
218     e.emp_name,
219     p.project_name,
220     ep.hours_worked
221 FROM Employee_Project ep
222 JOIN Employee e
223     ON ep.emp_id = e.emp_id
```

The results table displays the output of the query, sorted by hours_worked in descending order. The table has three columns: emp_name, project_name, and hours_worked. The data is as follows:

emp_name	project_name	hours_worked
Shreya Das	AI Security System	160
Dev Malhotra	Machine Learning Research	160
Rohan Verma	Cloud Migration	150
Varun Nair	AI Security System	150
Meera Shah	Financial Analysis	150
Nisha Reddy	Financial Analysis	145
Aditi Mishra	Machine Learning Research	140
Isha Malhotra	Digital Marketing	140
Arjun Patel	Financial Analysis	140
Kavya Iyer	Financial Analysis	135
Diya Kapoor	AI Security System	135
Riya Agarwal	Employee Training	130
Karan Mehta	AI Chatbot	130
Tanya Arora	Market Research	130